

TfL What-If Simulator

Testing, Evaluation and Quality Evidence

Assessment-focused testing and evaluation documentation

Document Structure

- 1. Purpose and scope**
- 2. Implemented testing evidence**
- 3. Manual test plan**
- 4. Questionnaire/user feedback evidence**
- 5. Quality evaluation**
- 6. Defect and improvement log**
- 7. Future testing**

This document uses an assessment-focused structure: summary first, implementation detail second, then rationale, limitations and evidence. This makes it suitable for project marking, demonstration preparation and final report support.

1. Purpose and Scope

This document explains how the TfL What-If Simulator can be evaluated for correctness, usability and quality. It combines the evidence already present in the project with a structured manual test plan suitable for demonstration or assessment. The aim is not to claim the product is production complete; it is to show that the major implemented behaviours can be verified and critically evaluated.

2. Implemented Testing Evidence

Test file	Area tested	Evidence of quality
__tests__/graph.test.js	Graph construction from nodes and edges.	Checks node initialisation, bidirectional edges, weights and multiple connections.
__tests__/pathfinding.test.js	Dijkstra routing behaviour.	Checks shortest path selection, no-path cases, same start/end station and closure avoidance.
frontend/__tests__/MapSearchBox.test.js	Search UI behaviour.	Checks Enter selects first match and clicking a suggestion calls the station select handler.
frontend/__tests__/StationLayer.test.js	Station closure marker rendering.	Checks live-closed station marker appears and does not appear for normal station.
frontend/__tests__/stationMarkerSizing.test.js	Marker sizing helpers.	Checks radius scaling, state boosts and occlusion radius logic.

The strongest existing testing evidence is around the algorithmic core and small UI behaviours. This is valuable because routing and marker interaction are high-risk areas: if those are wrong, the simulation becomes misleading. However, the project would benefit from more integration testing across the complete user journey.

3. Manual Test Plan

ID	Test	Steps	Expected result	Priority
TC-01	Load the map	Open the application.	Map loads with Underground stations/lines visible. Search is collapsed initially. No console-breaking error occurs.	High
TC-02	Search for a station	Open search, type a known station, click a result.	Map focuses the selected station and highlights/focuses it.	High
TC-03	Plan a basic route	Click a start station and destination station.	A route appears, route panel updates, route is highlighted on map.	High
TC-04	Route auto-fit	Select two distant stations.	Map zooms/pans so the full route is visible.	Medium
TC-05	Reset route/view	Create a route, then use reset/clear controls.	Route clears and map returns to a predictable default view/state.	Medium
TC-06	Enable What-If mode	Use the main-screen What-If toggle.	The app enters What-If mode, visual styling changes and closure controls become	High

			meaningful.	
TC-07	Close a station	In What-If mode, close a station and calculate a route that would normally use it.	Closed station is visually marked and routing avoids it where possible.	High
TC-08	Close a line	In What-If mode, close a line from the line controls.	Routes avoid that line and line status in the panel reflects closure.	High
TC-09	Save a scenario	Close at least one station/line and save the scenario with a name.	Scenario appears in saved list and persists after refresh in same browser.	High
TC-10	Load a scenario	Load a previously saved scenario.	Same closed stations and lines are restored. What-If mode is active.	High
TC-11	Delete a scenario	Delete a saved scenario.	Scenario disappears from saved list and is removed from localStorage.	Medium
TC-12	Live line status display	Leave What-If mode off.	Line status/delay indicators display live or fallback state without breaking UI.	Medium
TC-13	Train visualisation	Enable train display.	Train markers appear from live data or fallback simulation. Selected train panel works.	Medium
TC-14	Mobile portrait layout	Open on a mobile-size viewport.	Mobile controls are usable and do not permanently cover the map.	Medium
TC-15	UI overlap check	Switch between normal and What-If mode.	Search/toggle/camera controls stay aligned and do not overlap.	High

4. Questionnaire/User Feedback Evidence

A key evaluation finding from the project questionnaire was that many users had difficulty finding What-If mode. This is significant because What-If mode is the project's main differentiating feature. If users cannot discover it, the technical implementation is less valuable because its core behaviour is not accessible.

The design response was to move the What-If toggle from the sidebar onto the main map screen. This keeps the same mode-changing function but increases discoverability. The search panel was also made collapsed on first load to reduce clutter, and floating controls were aligned to avoid overlap. These changes show an iterative user-centred process: feedback was collected, a usability issue was identified, and the interface was revised without rewriting the underlying simulation logic.

Feedback issue	Design response	Quality benefit
Users struggled to find What-If mode.	Move the toggle to the main screen.	Improves discoverability of the core feature.
Initial UI could feel cluttered.	Load the search box collapsed.	Preserves map visibility on first load.
Controls overlapped in What-If mode.	Adjust positions based on active mode.	Improves visual clarity and prevents accidental obstruction.
Routes could be hard to see after selection.	Auto-fit camera to selected route.	Reduces manual zoom/pan effort.

5. Quality Evaluation

Functional correctness

The system implements the major required behaviours: station graph display, route selection, disruption simulation, line/station closures, scenario persistence and live service context. The presence of automated tests for graph and pathfinding behaviour strengthens confidence in the core logic.

Usability

The project improved over time through feedback-driven changes. Moving the What-If toggle to the main screen is a strong example of reacting to real user difficulty rather than only developer preference.

Robustness

The code includes fallbacks for live services, including fallback line definitions and fallback train animation. This is important because public APIs are not fully under the project team's control.

Maintainability

The project is divided into components, hooks and utility modules. This helps keep UI rendering, data fetching, route logic and simulation state more understandable than a single large file would be.

Performance

Caching the station graph and debouncing map state updates are sensible performance choices. More formal performance testing could still be added for large datasets or prolonged live train rendering.

6. Defect and Improvement Log

Observed problem	Likely cause	Resolution / recommendation
What-If mode was difficult to discover.	Toggle was hidden in sidebar.	Move toggle to main map screen.
Search UI and What-If controls could overlap.	Floating controls changed position between modes.	Fix consistent right-edge alignment and mode-aware offsets.
Routes sometimes required manual zooming.	Map camera did not react to route selection.	Use route bounds to auto-fit selected path.
Live train data may be unavailable.	TfL arrivals may be incomplete or API can fail.	Use fallback train movement and show feed status.
Saved scenarios are not portable.	localStorage is browser/device specific.	Future backend persistence or JSON export/import.

7. Future Testing

- Add end-to-end tests for the full route-planning journey.
- Add tests for saving/loading scenarios with mocked localStorage.
- Add visual regression checks for map control positions in normal and What-If modes.
- Add mocked TfL API tests for live status and train feed failure cases.

- Run structured usability testing after the What-If toggle redesign and compare results with the original questionnaire feedback.